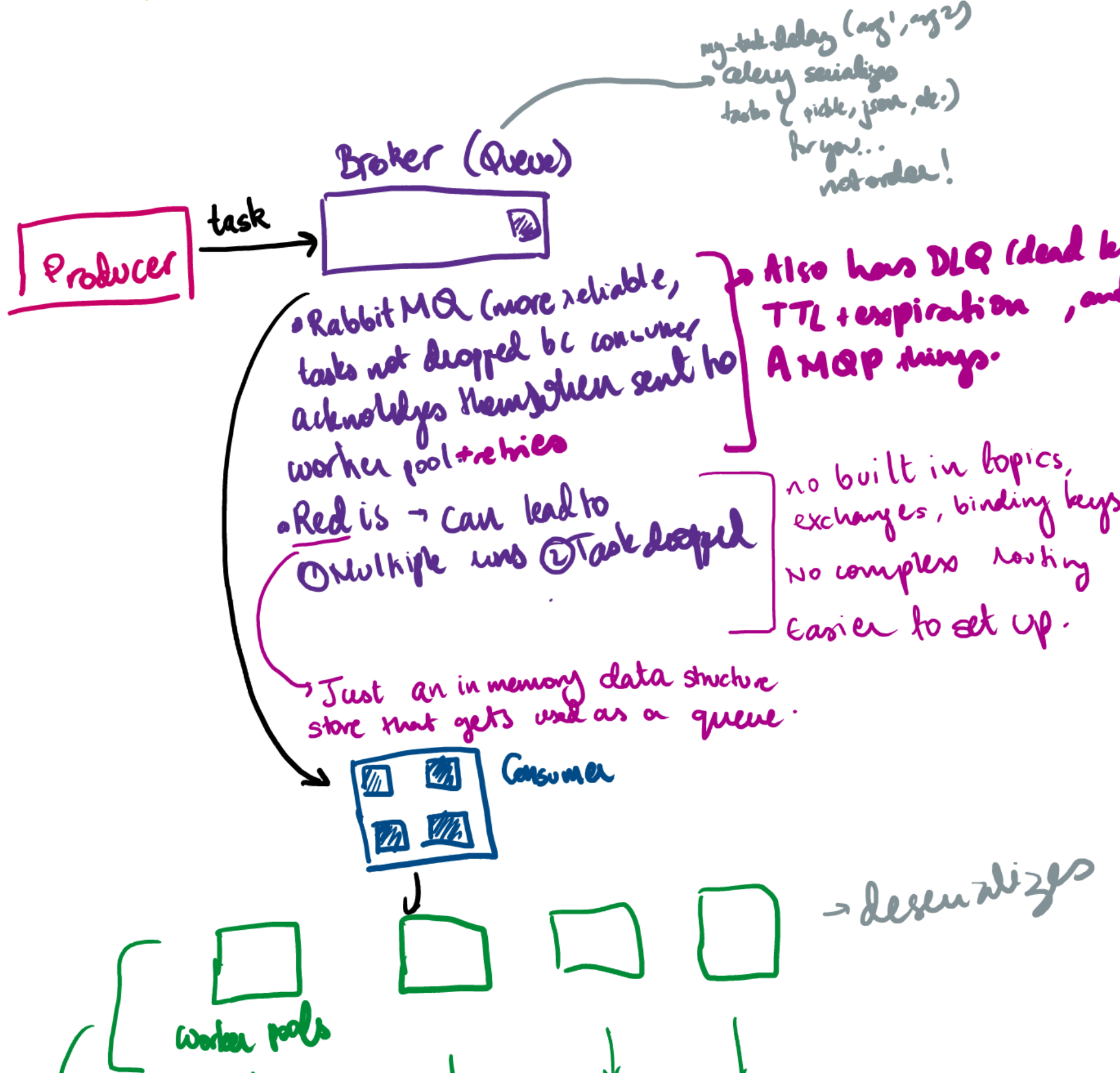
Celery

07 September 2024 18:50

- like spark, celery is distributed.
- But, spark works on datasets. Celery just ensures we run this task, somewhere else.



my_task.delay('arg1', arg2) → celery serializes tasks (pickle, json, etc.) for you... not order!

- RabbitMQ (more reliable, tasks not dropped bc consumer acknowledges them, then sent to worker pool + retries
  → Also has DLQ (dead letter queues), TTL + expiration, and other AMQP things.
- Redis → can lead to ① Multiple runs ② Task dropped
  no built in topics, exchanges, binding keys.
  No complex routing
  Easier to set up.
  → Just an in memory data structure store that gets used as a queue.

results backend (stores tasks results. Client/others can fetch result/status of tasks)

number defined by --concurrency.
→ If you need order, have a dedicated queue + 1 worker, or task chaining task1 | task2 | task3
broker will start task 2 only when
⇒ extra part of backend to broker to schedule task.

- green threads → managed by the user space, not OS; light weight.
  greenlets
  → a green thread must explicitly yield so another can run. OS threads don't have to do that; truly parallel.
  → while one thread is waiting for I/O, another can run.
  if CPU bound → use multiprocessing (not green threads)
  ↓
  Celery → prefork (default)

- Is ordering guaranteed if I use RabbitMQ | Redis?
  - Producer → Broker (Queue) ↳ has FIFO, order preserved.
    - if only one consumer OR
    - if multiple consumers but each message is processed synchronously & acked in order

yes for RabbitMQ, but if Redis consumer dies, message maybe lost.

→ Also this is the order in which it reaches the queue that is auto preserved, not the order in which it is sent!

- Depends on client + network protocol
  → TCP guarantees ordered delivery per connection, but if the client opens multiple connections, no guarantee.
  Client needs to guarantee order.

If you need strict ordering choose Kafka, and:
- use a single partition
- use a partition key
- use one producer connection
- enable idempotence + sequence guarantees (Kafka supports this)

but RabbitMQ can get close if single channel, single consumer.

- if many tasks tho, you can get lots of tasks into consumer w/ CELERY_PREFETCH_MULTIPLIER.
  ⇒ More things into consumer
  too small → can make tasks chunks ... only 5 tasks instead of 1.

Each command creates 1 worker instance:
Usually, you just create multiple workers, each with a different pool type.

```bash
celery -A app worker -n cpu@%h --pool=prefork --concurrency=8
celery -A app worker -n io@%h --pool=gevent --concurrency=500
```

→ for CPU bound tasks
→ for IO bound tasks.

& routes to tasks:

```python
task_routes = {
    'cpu_heavy_task': {'queue': 'cpu'},
    'io_task'       : {'queue': 'io'},
}
```

↳ pickle
→ python's object serialization mechanism.

celery uses pickle/JSON/msgpack to send task arguments from producer to worker.
Sometimes move data between processes.

(pickle can serialize "real python objects"
JSON can only represent certain types (dict/list/str/int/float...)
Pickle can handle datetime, Decimal, custom classes, exceptions, etc.
without you writing conversion code.
Pickle is not as secure tho ... not safe to accept from untrusted producers.
JSON: human readable, widely supported, but limited types
msgpack: smaller + faster than JSON for some payloads.

You can pick which serializer is used in Celery
- 'task_serializer' → specify what serializer you send tasks with
- 'accept_content' → what serializers workers will accept
- 'results_serializer' → what serializer is used for results.

☆ Green threads vs Multithreading
↓
→ equivalent to coroutines.
They are managed by the user space & not by OS kernel
A green thread runs until it explicitly yields.
One OS thread can have multiple green threads.

In celery, you can specify what kind of worker pool you want:
- prefork (default) → Multiprocessing → concurrency specifies number of processes. If one crashes, it doesn't effect other children. Context switches are v heavy bc each process has its own interpreter.
- eventlet → Green threads → One celery worker process will have many green threads, each running one task at a time. Green thread context switching is v cheap compared to OS thread switching. However, if a green thread does something blocking like time.sleep or call a non cooperative DB driver, it can stall the entire process, freezing all greenlets until it returns. So higher concurrency at the cost of needing cooperative IO.
- threads → Concurrency specifies number of OS threads within 1 process.
  ↳ python has a GIL (Global Interpreter Lock). So only one thread can execute python byte code at a time. Threads need to release the GIL on a switch (same for green threads).
  This doesn't apply in multiprocessing tho ... there would be PARALLEL execution on multiple cores
  ↳ better for CPU heavy
  better for I/O bound

Also processes don't share memory ⇒ so you need to serialize python objects so parent & child can share.